

Contents

Exercise 2 — Build the pipeline, then break it on purpose	1
Goal	1
Dataset	1
Tasks	2
1. Look, before anything else	2
2. Split, before anything else	2
3. Outliers	2
4. Missing values	2
5. The honest pipeline	2
6. Build a leak on purpose	2
7. Explain the gap	3
8. Conclude	3
What to hand in	3
Assessment criteria	3
Hints	4

Exercise 2 — Build the pipeline, then break it on purpose

Set: Friday 2 October 2026 · **Due:** Friday 9 October 2026, start of Lesson 3

Roughly 2–3 hours. Work alone; discussing ideas with others is fine, sharing notebooks is not.

Goal

Build a complete, leak-free preparation pipeline for a messy dataset, then — this is the part that carries the most marks — **deliberately construct a leak, measure what it manufactures, and fix it**. Anyone can follow the rule “scale inside the pipeline” by rote. This exercise checks that you understand *why*, by having you break it on purpose and see the number it produces.

Dataset

The same synthetic telecom churn dataset used in the lesson, from a different random seed so your numbers will not match anyone else’s. Copy `churn_data.py` — from this lesson’s `Notebooks/` folder — into the same folder as your submitted notebook, then:

```
from churn_data import make_churn_data
```

```
df = make_churn_data(n=2000, seed=104)
```

Keep the seed fixed at 104 so your results are reproducible and your markdown commentary matches the numbers your code actually produces.

The columns are the same as in the lesson: numeric (`tenure_months`, `monthly_charges`, `age`, `num_support_calls`), low-cardinality categorical (`contract_type`, `region`), a high-cardinality cat-

egorical (`zip_code`), and the target `churned`. Nothing about their meaning has changed — only the random draw.

Tasks

1. Look, before anything else

Report the shape, the dtypes, the churn rate and the resulting baseline accuracy. Report which columns have missing values and by how much. For each, state whether you believe it is MCAR or MAR, and what observation led you to that judgement (you do not have access to the generating code's internals — argue from what you can observe, the way you would with a real dataset).

2. Split, before anything else

Hold out a test set. State your proportion and whether you stratified, and why.

3. Outliers

Using both the z-score and IQR rules from the lesson, find candidate outliers in `tenure_months` and `monthly_charges`. Then apply **at least one domain rule** that neither statistical rule would catch on its own, and explain what it catches that the statistical rules miss.

4. Missing values

Choose and justify a treatment for each column with missing values. If you use an indicator column for a MAR column, say why. Whatever you choose, it must be **fitted on the training fold only** — say in one sentence where in your code that is enforced.

5. The honest pipeline

Build a single `ColumnTransformer` + `Pipeline` that imputes, scales the numeric columns, encodes the low-cardinality categoricals, and fits a `LogisticRegression`. Do **not** include `zip_code` yet. Report baseline accuracy, model accuracy, and model AUC on the test set, and explain in one or two sentences why AUC is the more informative number here.

6. Build a leak on purpose

Pick **one** of the following and implement it exactly as described — the “wrong” way, deliberately:

- **(a) Leaky imputation.** Impute the numeric columns' missing values with `KNNImputer`, fitted on the concatenation of your training and test data, *before* using the result to train and evaluate a model.
- **(b) Leaky encoding.** Encode `zip_code` by the mean churn rate of its members, computed over the **whole** dataset (train and test together), and add it as a feature before training and evaluating a model.

Report the resulting test AUC. Then build the honest version of the same step (imputation fitted on the training fold only inside the pipeline; or `sklearn.preprocessing.TargetEncoder` fitted inside the pipeline for the encoding option) and report its test AUC alongside the leaky one.

7. Explain the gap

In your own words, explain *why* the two numbers in Task 6 differ, using the mechanism from the handout (Section 9.1 for imputation, Section 9.2 for encoding) rather than just restating “it leaked.” If you chose the encoding option, find at least one specific `zip_code` value in your data for which the leak is especially large or especially small, and explain why using the category-size argument from Section 9.2.

8. Conclude

In no more than 150 words: which of your preprocessing decisions were you least sure about, and what would you check first if you had another day with this dataset?

What to hand in

A single notebook named `{surname}_02.ipynb` that:

- runs top to bottom in the course container with no edits beyond the file path to `churn_data.py`
 - has `random_state` fixed everywhere, so your numbers reproduce
 - keeps its outputs, so it can be read without being run
 - contains **both** the leaky and the honest version of your chosen Task 6 step, side by side, with their scores clearly labelled
-

Assessment criteria

Criterion	Weight	What earns marks
Methodological correctness	40%	Split before anything is learned; every fitted step lives inside the pipeline in the honest version; the domain-rule outlier check is genuinely independent of the statistical ones
Implementation	30%	The leak is implemented <i>exactly</i> as specified (not a weaker version of it); both the leaky and honest numbers are reported; code is clean and reproducible
Interpretation and communication	30%	Task 7’s explanation uses the actual mechanism, not just the word “leakage”; the missingness-mechanism judgement in Task 1 is argued from evidence; Task 8 is honest about uncertainty

There are **no marks for how large or small the leak turns out to be** — a small, correctly explained gap earns full marks; a large, unexplained one does not. As in Exercise 1, a submission that leaves a fitted step outside the pipeline in the *honest* version loses methodology marks regardless of its scores.

Hints

- `df.groupby("zip_code")["churned"].mean()` is the one-line version of the leaky encoding in Task 6(b) — resist the urge to make it more elaborate than the handout’s description; the exercise is about seeing the ordinary, unremarkable version of the bug.
- For the honest version of 6(b), `sklearn.preprocessing.TargetEncoder` inside your `ColumnTransformer` handles the cross-fitting for you; you do not need to implement leave-one-out by hand, though you are welcome to.
- Task 7’s “specific `zip_code` value” is easiest to find by sorting the value counts and looking at the least frequent codes — the notebook’s singleton-code example is a template for the argument, not something to reuse verbatim on data with a different seed.
- If your chosen option’s leak looks tiny, do not “fix” the dataset to make it bigger — report the honest number and explain, per handout Section 9.1, that a leak not being dramatic does not make it valid.